Lesson 2: Smart Contract Introduction & Basic Tooling

What We Will Do Today

1. Install `sc-meta` and scaffold a contract
2. Build and test it locally
3. Investigate the build output
4. Explore the project structure
5. Deploy and interact using snippets and the interactor

The Tool: `sc-meta`

`sc-meta` is the official MultiversX smart contract meta-tool.

It handles:

- scaffolding new contracts
- building and compiling
- deploying and calling contracts on any network
- managing ABIs and proxies

One tool for the full lifecycle.

Step 1: Install `sc-meta`

```
cargo install multiversx-sc-meta --locked
```

Verify it works:

```
sc-meta --version
```

This installs the `sc-meta` binary into `~/.cargo/bin`.

Step 2: Create a New Contract

```
sc-meta new --template adder --name my-adder
```

- `--template adder` — start from the official adder example
- `--name my-adder` — names the contract and Rust crate `my-adder`

This generates a complete, working project folder: `my-adder/`

Step 3: Check and Test

```
cd my-adder
cargo check
cargo test
```

`cargo check` — analyzes types and imports, no binary produced.

`cargo test` — runs all test suites in the project.

Expected: no errors, all tests pass.

Step 4: Build the Contract

```
sc-meta all build
```

Compiles to WebAssembly and generates the build artifacts in `output/`:

```
output/
├── my-adder.wasm          ← the compiled contract bytecode
├── my-adder.abi.json      ← the contract ABI
├── my-adder.imports.json ← WASM imports
└── my-adder.mxsc.json     ← ABI + WASM bundled together
```

WebAssembly

Smart contracts on MultiversX run as **WebAssembly (WASM)** bytecode.

- WASM is a compact binary format designed for safe sandboxed execution
- The blockchain VM executes `my-adder.wasm` on every call
- The contract has no OS access — it can only call VM host functions

The `wasm/` crate is auto-generated. It wires the contract endpoints to the WASM exports:

Reading the Assembly

```
sc-meta all build --wat
```

Produces `output/my-adder.wat` — the human-readable text format of the WASM binary.

```
(module
  (import "env" "mBufferStorageStore" (func ...))
  (import "env" "bigIntAdd"           (func ...))
  ...
  (export "add"    (func 17))
  (export "getSum" (func 19))
  (export "init"   (func 20))
)
```

The `import "env" ...` lines are the VM host functions the contract calls into. Useful for auditing what the contract actually does at the binary level.

What Is An ABI?

ABI = **Application Binary Interface**

It describes the contract's public API in a machine-readable format:

- endpoint names and arguments
- view functions
- events
- constructor parameters

```
{
  "endpoints": [
    { "name": "add",    "inputs": [{ "type": "BigUint" }] },
    { "name": "getSum", "outputs": [{ "type": "BigUint" }] }
  ]
}
```

The `.mxsc.json` File

`my-adder.mxsc.json` bundles the ABI and the WASM bytecode into one file.

- This is what gets deployed to the chain
- Wallets and tools use the ABI to encode/decode calls
- Explorers use the ABI to show human-readable transaction data

When you deploy, you hand over this file.

The Project Layout

```
my-adder/
├── src/            ← contract logic
├── tests/          ← automated tests
├── interactor/     ← live network interaction tool
├── snippets/       ← shell scripts for deployment
├── meta/           ← build metadata helper crate
├── wasm/           ← WASM output crate, auto-generated
├── output/         ← build artifacts (after sc-meta all build)
└── sc-config.toml
```

Break

`src/` — The Contract

```
src/
├── my_adder.rs           ← the contract trait
└── my_adder_proxy.rs     ← auto-generated proxy (do not edit)
```

The Contract: `my_adder.rs`

```rust
#[multiversx_sc::contract]
pub trait MyAdder {
    #[view(getSum)]
    #[storage_mapper("sum")]
    fn sum(&self) -> SingleValueMapper<BigUint>;

    #[init]
    fn init(&self, initial_value: BigUint) {
        self.sum().set(initial_value);
    }

    #[endpoint]
    fn add(&self, value: BigUint) {
        self.sum().update(|sum| *sum += value);
    }
}
```

Contract Annotations

Annotation	Meaning
`#[multiversx_sc::contract]`	marks the trait as a smart contract
`#[init]`	called once on deploy
`#[upgrade]`	called on upgrade
`#[endpoint]`	callable by anyone via a transaction
`#[view]`	read-only query, no state change
`#[storage_mapper]`	persistent on-chain storage

The Proxy: `my_adder_proxy.rs`

```
// Code generated by the multiversx-sc proxy generator. DO NOT EDIT.
```

- Auto-generated from the contract ABI
- Provides a typed Rust API for calling the contract
- Used by tests and the interactor
- Regenerated with `sc-meta all proxy`

You read it; you never write it.

`tests/` — Automated Tests

```
tests/
├── my_adder_unit_test.rs          ← pure Rust, no blockchain
├── my_adder_blackbox_test.rs      ← simulated chain, typed API
├── my_adder_whitebox_test.rs      ← simulated chain, internal access
└── my_adder_scenario_rs_test.rs ← scenario files driven from Rust
```

Four flavors of testing, from fastest to most realistic.

Unit Test

```
#[test]
fn adder_unit_test() {
    let adder = my_adder::contract_obj::<SingleTxApi>();
    adder.init(BigUint::from(5u32));
    assert_eq!(BigUint::from(5u32), adder.sum().get());
    adder.add(BigUint::from(7u32));
    assert_eq!(BigUint::from(12u32), adder.sum().get());
}
```

- No simulated blockchain at all
- Calls contract methods directly as Rust functions
- Fastest to run, great for pure logic

Blackbox Test

```
world
    .tx()
    .from(OWNER_ADDRESS).to(ADDER_ADDRESS)
    .typed(my_adder_proxy::MyAdderProxy)
    .add(1u32)
    .run();

world
    .query()
    .to(ADDER_ADDRESS)
    .typed(my_adder_proxy::MyAdderProxy)
    .sum()
    .returns(ExpectValue(6u32))
    .run();
```

- Simulated blockchain; uses the proxy — same API as real calls
- Tests the contract as a user would

`snippets/` — Shell Scripts

```
snippets/my_adder.snippets.sh
```

Shell functions that wrap `sc-meta tx` commands:

```
deploy()   # deploy the contract to the chain
add()      # call the add endpoint
getSum()   # query the current sum
upgrade()  # upgrade the contract
```

Good for quick manual operations without writing Rust.

Live Demo: Snippets

```
# 1. Generate a test wallet
sc-meta wallet test-wallet --name alice

# 2. Edit the script: set NETWORK=devnet

# 3. Deploy
source snippets/my_adder.snippets.sh && deploy

# 4. Call add
add

# 5. Query
getSum
```

After each step: check the transaction in the Devnet Explorer.

Use Your Own Wallet

The template uses a built-in test wallet (`alice`).

Replace it with your own:

```
sc-meta wallet new --format pem --outfile my-wallet.pem
```

In `my_adder.snippets.sh` :

```
ALICE="my-wallet.pem"   # was: "alice.pem"
```

Fund it from the Devnet Wallet (as we did in Lesson 1), then run `deploy` again.

`interactor/` — Rust Interaction Tool

A standalone Rust binary that talks to a real (or simulated) network.

```
interactor/
├── src/
│   ├── basic_interactor.rs        ← deploy, add, query logic
│   ├── basic_interactor_cli.rs    ← CLI argument parsing
│   ├── basic_interactor_config.rs ← network config
│   └── basic_interactor_state.rs  ← persists deployed address
├── config.toml                    ← gateway, chain type
└── state.toml                     ← saved contract address
```

Fix the Interactor Before Running

The template was generated for the upstream `adder` crate.

Two things to fix:

1. Fix the bytecode path (`src/basic_interactor.rs`):

```
// before (template default):
const ADDER_CODE_PATH: MxscPath = MxscPath::new("../output/adder.mxsc.json");

// after:
const ADDER_CODE_PATH: MxscPath = MxscPath::new("../output/my-adder.mxsc.json");
```

2. Remove the stale workspace override (same file):

```
// delete this line:
interactor.set_current_dir_from_workspace("contracts/examples/adder/interactor");
```

`config.toml` — Network Config

```
# chain_type = 'simulator'
# gateway_uri = 'http://localhost:8085'

chain_type = 'real'
gateway_uri = 'https://devnet-gateway.multiversx.com'
```

Switch between devnet, testnet, or a local chain simulator here.

`state.toml` — Persisted Address

```
adder_address = "erd1qqqqqqqqqqqqqpgq..."
```

After `cargo run deploy`, the interactor saves the deployed address here.

Subsequent `add` and `sum` calls read from this file automatically.

Live Demo: Interactor

```
cd interactor

# deploy the contract
cargo run deploy

# call add with a value
cargo run add --value 7

# query the current sum
cargo run sum
```

Uses the same wallet registered in the code (`test_wallets::mike`).

Check the transaction in the Devnet Explorer.

Snippets vs Interactor

	Snippets	Interactor
Language	Bash	Rust
Wallet	PEM file	registered in code
Good for	quick calls, scripting	full programmatic control
State management	`sc-meta.data-storage.json`	`state.toml`
Error handling	basic	full Rust

Summary

```
cargo install multiversx-sc-meta
sc-meta new --template adder --name my-adder
cd my-adder
cargo check && cargo test
sc-meta all build           # → output/my-adder.mxsc.json
```

Deploy with snippets or the interactor.

Inspect every transaction in the Devnet Explorer.

Homework

Deploy an `adder` contract using **your own wallet** on Devnet.

Goal:

Make the stored sum equal the number of letters in your first name.

(e.g. "Alice" → 5 calls of add 1, or one call of add 5)

Steps:

1. `sc-meta wallet new --format pem --outfile my-wallet.pem`
2. Fund it from the Devnet Faucet
3. Deploy using snippets or the interactor
4. Call `add` until `getSum` returns the right number

Submission:

Post your contract address in the Telegram group or Google Classroom.